CCIT Report #536
May 2005

Formal Verification of Synchronizers

Tsachy Kapschitz and Ran Ginosar

VLSI Systems Research Center, Electrical Engineering Department
Technion–Israel Institute of Technology, Haifa 32000, Israel
[ran@ee.Technion.ac.il]

Abstract. Large Systems on Chips (SoC) comprise multiple clock domains, and inter-domain data transfers require synchronization. Synchronizers may fail due to metastability, but when using proper synchronization circuits the probability of such failures can be made negligible. Failures due to unexpected order of events (caused by interfacing multiple unrelated clocks) are more common. Correct synchronization is independent of event order, and can be verified by model checking. Given a synchronizer, a correct protocol is guessed, verification rules are generated out of the protocol specification, and the model checker applies these rules to the given synchronizer. An alternative method verifies correct data transfer and seeks potential data missing or duplication. Both approaches require specific modeling of multiple clocks, allowing for non-determinism in their relative ordering. These methods have been applied successfully to a two-flip-flop synchronizer and a dual clock FIFO.

1 Introduction

Synchronous (clocked) hardware systems are typically partitioned into multiple clock domains. All sequential elements in the same clock domain are clocked at presumably the exact same time, but the clocks at different domains may be mutually unrelated—they may operate at different frequencies, and even when operating at the same frequency they may tick at different times. These relative frequency and phase differences may be unknown a-priori, and may also change over time [1]. Such multi-clock domain systems are also termed GALS (globally asynchronous, locally synchronous) since the different clocks are assumed mutually asynchronous.

Data transfers between different clock domains require synchronization [2]. Data that enters a domain and happens to change exactly when the receiving register is sampling its input may cause that register to become metastable and fail [3]. This problem is mitigated by properly employing synchronizers and by formally verifying their correctness. This paper describes methods for formal verification of synchronizers based on model-checking [4].

Common synchronizers comprise two levels, continuous and discrete. In the realm of continuous time, it is recognized that often an input change coincides with the sampling time and consequently the sampling circuit (typically a flip-flop; this paper does not treat latch-based synchronizers) may take an arbitrarily long time to resolve [3]. The synchronization circuit is allowed a certain time period S for resolution, so that (in case the input indeed changes state at the sampling point) the probability that the circuit fails to resolve S time later is negligible. However, when the